

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

MYSQL IMPLEMENTATION:

Creating Database and Tables:

```
1 • create database retail_project;
2
3 use retail_project;
4
5
```

Inserted Product and Sales Data:

	product_id	product_name	category	price	cost
▶	1	Laptop	Electronics	50000	45000
	2	Mouse	Electronics	700	300
	3	Keyboard	Electronics	1000	700
	4	Shoes	Fashion	2000	1500
	5	Tshirt	Fashion	800	500
*	NULL	NULL	NULL	NULL	NULL

	sale_id	product_id	store_id	quantity	sale_date
▶	1	1	101	2	2026-05-01
	2	2	101	5	2026-05-01
	3	4	102	3	2026-05-02
	4	3	102	4	2026-05-02
	5	5	103	6	2026-05-03
*	NULL	NULL	NULL	NULL	NULL

Stored Procedure Execution:

	store_name	total_sales
▶	Reliance Store	103500

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

MONGODB IMPLEMENTATION:

MongoDB Feedback Data Insertion:

```
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a06c5d383ab9a614d88e0d6'),
    '1': ObjectId('6a06c5d383ab9a614d88e0d7'),
    '2': ObjectId('6a06c5d383ab9a614d88e0d8')
  }
}
```

MongoDB Collection Data:

```
> db.campaign_feedback.find()
< {
  _id: ObjectId('6a059d8037b29f4e4c8d12cb'),
  customer_name: 'Rahul',
  product: 'Laptop',
  feedback: 'Good product',
  rating: 4
}
{
  _id: ObjectId('6a059d8037b29f4e4c8d12cc'),
  customer_name: 'Anjali',
  product: 'Shoes',
  feedback: 'Nice quality',
  rating: 5
}
{
  _id: ObjectId('6a059d8037b29f4e4c8d12cd'),
  customer_name: 'Kiran',
  product: 'Mouse',
  feedback: 'Average',
  rating: 3
}
{
  _id: ObjectId('6a06a92f83ab9a614d88e0d3'),
  customer_name: 'Rahul',
  product: 'Laptop',
  feedback: 'Good product',
  rating: 4
}
```

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

```
{
  _id: ObjectId('6a06a92f83ab9a614d88e0d4'),
  customer_name: 'Anjali',
  product: 'Shoes',
  feedback: 'Nice quality',
  rating: 5
}
{
  _id: ObjectId('6a06a92f83ab9a614d88e0d5'),
  customer_name: 'Kiran',
  product: 'Mouse',
  feedback: 'Average',
  rating: 3
}
{
  _id: ObjectId('6a06c5d383ab9a614d88e0d6'),
  customer_name: 'Rahul',
  product: 'Laptop',
  feedback: 'Good product',
  rating: 4
}
{
  _id: ObjectId('6a06c5d383ab9a614d88e0d7'),
  customer_name: 'Anjali',
  product: 'Shoes',
  feedback: 'Nice quality',
  rating: 5
}
```

```
{
  _id: ObjectId('6a06c5d383ab9a614d88e0d6'),
  customer_name: 'Rahul',
  product: 'Laptop',
  feedback: 'Good product',
  rating: 4
}
{
  _id: ObjectId('6a06c5d383ab9a614d88e0d7'),
  customer_name: 'Anjali',
  product: 'Shoes',
  feedback: 'Nice quality',
  rating: 5
}
{
  _id: ObjectId('6a06c5d383ab9a614d88e0d8'),
  customer_name: 'Kiran',
  product: 'Mouse',
  feedback: 'Average',
  rating: 3
}
```

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

PYTHON IMPLEMENTATION:

Python Script Execution:

```
SANTHOSH SELVA@DESKTOP-G5J7BLI MINGW64 /e/Employee Attendance and Productivity Tracker
$ python python/sales_analysis.py
sale_id product_id store_id quantity sale_date
0 1 1 101 2 2026-05-01
1 2 2 101 5 2026-05-01
2 3 4 102 3 2026-05-02
3 4 3 102 4 2026-05-02
4 5 5 103 6 2026-05-03

sale_id product_id store_id quantity sale_date ... cost store_name city revenue profit
0 1 1 101 2 2026-05-01 ... 45000 Reliance Store Mumbai 100000 10000
1 2 2 101 5 2026-05-01 ... 300 Reliance Store Mumbai 2500 1000
2 3 4 102 3 2026-05-02 ... 1500 Dmart Pune 6000 1500
3 4 3 102 4 2026-05-02 ... 700 Dmart Pune 4000 1200
4 5 5 103 6 2026-05-03 ... 500 Big Bazaar Delhi 4800 1800
5 6 1 103 1 2026-05-03 ... 45000 Big Bazaar Delhi 50000 5000

[6 rows x 13 columns]
revenue profit
store_name
Big Bazaar 54800 6800
Dmart 10000 2700
Reliance Store 102500 11000

sale_id product_id store_id quantity sale_date ... cost store_name city revenue profit
0 1 1 101 2 2026-05-01 ... 45000 Reliance Store Mumbai 100000 10000
2 3 4 102 3 2026-05-02 ... 1500 Dmart Pune 6000 1500
5 6 1 103 1 2026-05-03 ... 45000 Big Bazaar Delhi 50000 5000

[3 rows x 13 columns]
file saved
```

Generated Output CSV File:

```
output > final_sales.csv > data
1 sale_id,product_id,store_id,quantity,sale_date,product_name,category,price,cost,store_name,city,revenue,profit
2 1,1,101,2,2026-05-01,Laptop,Electronics,50000,45000,Reliance Store,Mumbai,100000,10000
3 2,2,101,5,2026-05-01,Mouse,Electronics,500,300,Reliance Store,Mumbai,2500,1000
4 3,4,102,3,2026-05-02,Shoes,Fashion,2000,1500,Dmart,Pune,6000,1500
5 4,3,102,4,2026-05-02,Keyboard,Electronics,1000,700,Dmart,Pune,4000,1200
6 5,5,103,6,2026-05-03,Tshirt,Fashion,800,500,Big Bazaar,Delhi,4800,1800
7 6,1,103,1,2026-05-03,Laptop,Electronics,50000,45000,Big Bazaar,Delhi,50000,5000
```

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

PYSPARK IMPLEMENTATION:

```
SANTHOSH SELVA@DESKTOP-G5J7BLI MINGW64 /e/Retail Sales Performance Dashboard (main)
• $ python python/sales_analysis.py
sale_id product_id store_id quantity sale_date
0 1 1 101 2 2026-05-01
1 2 2 101 5 2026-05-01
2 3 4 102 3 2026-05-02
3 4 3 102 4 2026-05-02
4 5 5 103 6 2026-05-03

sale_id product_id store_id quantity sale_date product_name ... price cost store_name city revenue profit
0 1 1 101 2 2026-05-01 Laptop ... 50000 45000 Reliance Store Mumbai 100000 55000
1 2 2 101 5 2026-05-01 Mouse ... 500 300 Reliance Store Mumbai 2500 2200
2 3 4 102 3 2026-05-02 Shoes ... 2000 1500 Dmart Pune 6000 4500
3 4 3 102 4 2026-05-02 Keyboard ... 1000 700 Dmart Pune 4000 3300
4 5 5 103 6 2026-05-02 Tshirt ... 800 500 Big Bazaar Delhi 4800 4300
5 6 1 103 1 2026-05-03 Laptop ... 50000 45000 Big Bazaar Delhi 50000 5000

[6 rows x 13 columns]
revenue profit
store_name
Big Bazaar 54800 9300
Dmart 10000 7800
Reliance Store 102500 57200
sale_id product_id store_id quantity sale_date product_name ... price cost store_name city revenue profit
1 2 2 101 5 2026-05-01 Mouse ... 500 300 Reliance Store Mumbai 2500 2200
2 3 4 102 3 2026-05-02 Shoes ... 2000 1500 Dmart Pune 6000 4500
3 4 3 102 4 2026-05-02 Keyboard ... 1000 700 Dmart Pune 4000 3300
4 5 5 103 6 2026-05-03 Tshirt ... 800 500 Big Bazaar Delhi 4800 4300

[4 rows x 13 columns]
File saved
```

PySpark Store-Level Analysis:

```
data = sales.join(products,"product_id")
data = data.withColumn(
    "total",
    col("quantity") * col("price")
)
data.show()
```

```
... +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|sale_id|store_id|quantity| sale_date|product_name| category|price| cost| total|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|          |1|      |6|    103| 1|2026-05-03| Laptop|Electronics|50000|45000| 50000|
|          |1|      |1|    101| 2|2026-05-01| Laptop|Electronics|50000|45000|100000|
|          |2|      |2|    101| 5|2026-05-01| Mouse|Electronics| 500| 300| 2500|
|          |3|      |4|    102| 4|2026-05-02| Keyboard|Electronics| 1000| 700| 4000|
|          |4|      |3|    102| 3|2026-05-02| Shoes| Fashion| 2000| 1500| 6000|
|          |5|      |5|    103| 6|2026-05-03| Tshirt| Fashion| 800| 500| 4800|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

```
top = data.groupBy("product_name") \
    .agg(sum("total").alias("sales"))
top.show()
```

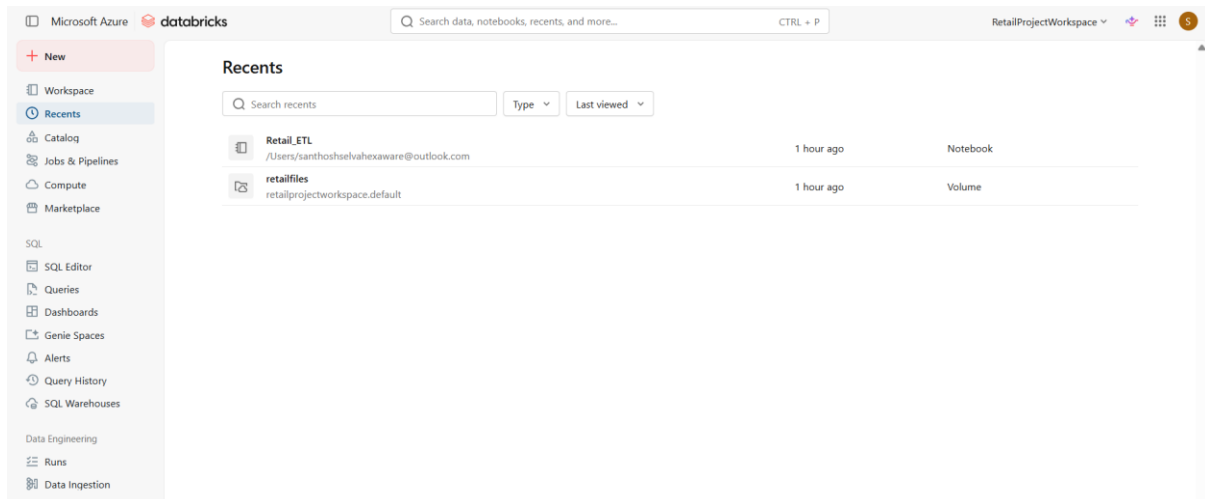
```
... +-----+-----+
|product_name| sales|
+-----+-----+
|      Laptop|150000|
|       Mouse| 25000|
|       Tshirt| 48000|
|        Shoes| 60000|
|    Keyboard| 40000|
+-----+-----+
```

OUTPUT FILE:

```
output > sales_summary.csv > data
1 store_name,revenue,profit
2 Big Bazaar,54800,9300
3 Dmart,10000,7800
4 Reliance Store,102500,57200
```

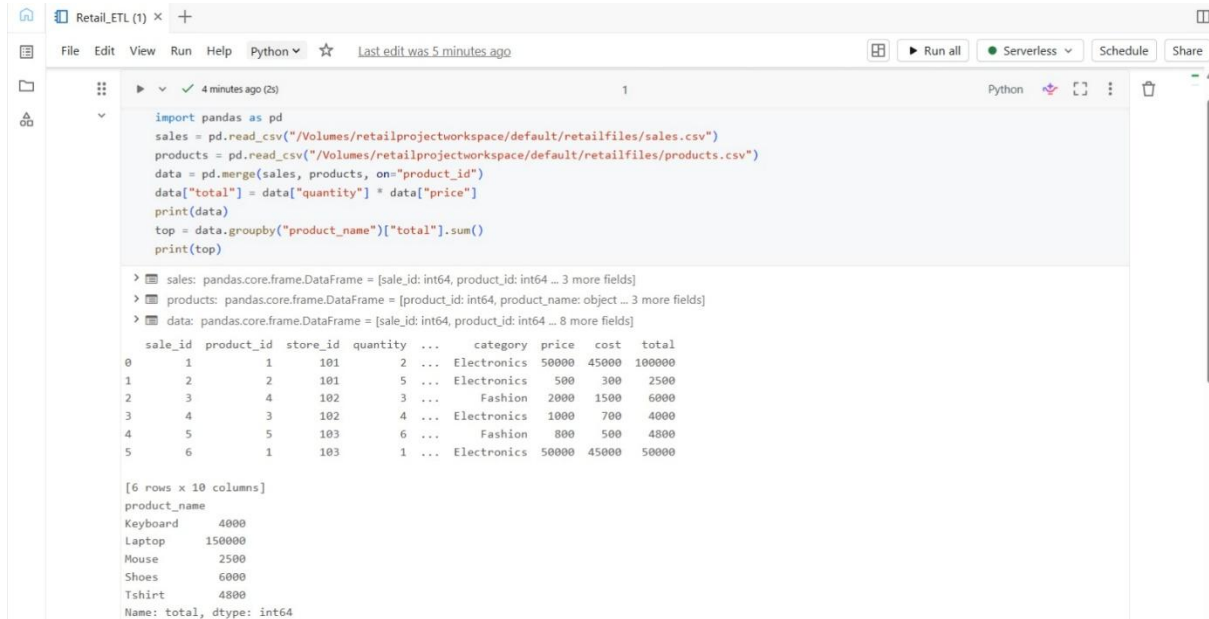
AZURE DATABRICKS IMPLEMENTATION:

AZURE DATABRICKS ETL PROCESS:



Retail Sales Performance Dashboard

- By Santhosh Kumar V S



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor. The code imports pandas and reads two CSV files: 'sales.csv' and 'products.csv'. It merges them on 'product_id', calculates a 'total' column as 'quantity * price', and groups the data by 'product_name' to find the top products. The output displays the merged data as a DataFrame and a summary of the top products.

```
import pandas as pd
sales = pd.read_csv("/Volumes/retailprojectworkspace/default/retailfiles/sales.csv")
products = pd.read_csv("/Volumes/retailprojectworkspace/default/retailfiles/products.csv")
data = pd.merge(sales, products, on="product_id")
data["total"] = data["quantity"] * data["price"]
print(data)
top = data.groupby("product_name")["total"].sum()
print(top)
```

Output:

```
> sales: pandas.core.frame.DataFrame = [sale_id: int64, product_id: int64 ... 3 more fields]
> products: pandas.core.frame.DataFrame = [product_id: int64, product_name: object ... 3 more fields]
> data: pandas.core.frame.DataFrame = [sale_id: int64, product_id: int64 ... 8 more fields]
```

sale_id	product_id	store_id	quantity	...	category	price	cost	total
0	1	1	101	2	Electronics	50000	45000	100000
1	2	2	101	5	Electronics	500	300	2500
2	3	4	102	3	Fashion	2000	1500	6000
3	4	3	102	4	Electronics	1000	700	4000
4	5	5	103	6	Fashion	800	500	4800
5	6	1	103	1	Electronics	50000	45000	50000

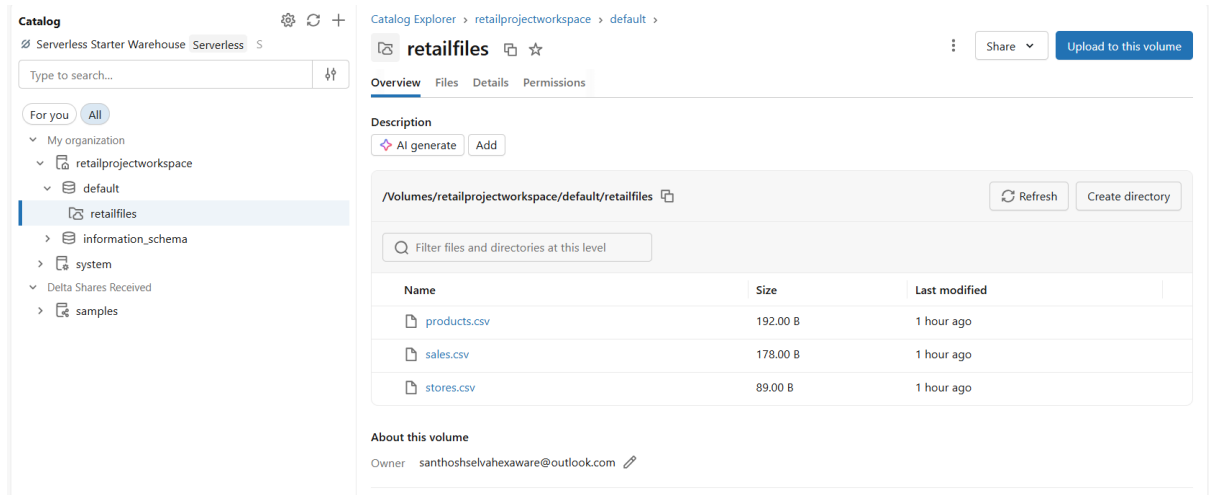
[6 rows x 10 columns]

product_name

product_name	total
Keyboard	4000
Laptop	150000
Mouse	2500
Shoes	6000
Tshirt	4800

Name: total, dtype: int64

Uploaded CSV Files in Databricks:



AZURE DEVOPS IMPLEMENTATION:

Retail Sales Performance Dashboard

- By Santhosh Kumar V S

AZURE DEVOPS PIPELINE:

Azure DevOps Pipeline Configuration

The screenshot shows the configuration page for an Azure DevOps pipeline. At the top, there's a header with the pipeline ID and name, a 'Run new' button, and a retention notice. Below this, the 'Summary' tab is active, showing details about the individual CI run by Santhosh Kumar, including the repository (RetailSalesProject), branch (main), and commit hash (af0fac3b). It also shows the number of changes (4) and parameters. The 'Stages' tab is also visible, showing a single stage with one job completed in 8 seconds. A 'Rerun Stage' button is present at the bottom of the stages section.

Azure DevOps Pipeline Execution:

The screenshot shows the execution log for a job named 'Job'. On the left, a table lists the jobs in the run, including 'Initialize job', 'Checkout RetailSalesP...', 'Run Retail Pipeline', 'Post-job: Checkout Re...', 'Finalize Job', and 'Report build status'. The 'Job' is highlighted. On the right, the raw log for the job is displayed, showing the pool (Azure Pipelines), image (ubuntu-latest), agent (Azure Pipelines 1), start time (Today at 12:18 PM), and duration (1s). The log also shows the job preparation parameters.